

CS2610: Computer Organization & Architecture Lab

Lab #8: Virtual Memory Management

Motivation:

In modern computer systems, virtual memory is a critical feature that enables efficient use of physical memory while providing process isolation and flexibility. In the RISC-V architecture, this is achieved through a hierarchical page table structure and the Supervisor Address Translation and Protection (SATP) mechanism.

Objective:

In this assignment, you will delve into the workings of virtual memory by creating and configuring page tables, enabling paging, and verifying address translations. This hands-on experience will help you understand the foundational concepts of memory management and privilege modes in RISC-V, preparing you to design and debug systems that rely on virtual memory.

Overview of the Task:

Implement a program in RISC-V assembly that does address virtualization with Sv48 as the addressing mode.

References:

1. [The RISC-V Instruction Set Manual, Volume II: Privileged Architecture](#)
 - Section 4.1 - Supervisor CSRs
 - Section 4.3.2 - Virtual Address Translation Process
 - Section 4.5 - Sv48: Page based 48-bit Virtual Memory System
 - Section 3.1 - Machine-Level CSRs
2. Lecture Slides: Virtual Memory, Privileges and Interrupts
3. [Shakti RISC-V Programmer's Manual](#)
4. [Spike Debugging Manual](#) , Alternatively use `help` command inside spike.
5. [RISC-V Assembly Documentation](#) , there are other cheat sheets and fragmented documentation available in the web as well.

Compilation:

For compilation, use

```
riscv64-unknown-elf-gcc -nostartfiles -T linker.ld Lab6_template.S
```

For running the executable without the proxy kernel, use

```
spike -d <executable>
```

To create a dump file, use

```
riscv64-unknown-elf-objdump -D <executable> > dump
```

Description of the Task:

1. The program should start from **Machine Mode**.
2. It should then switch its execution from **Machine Mode** to **Supervisor Mode** for address virtualization.
3. In the **Supervisor Mode**, the program should appropriately initialize the page table and correspondingly configure the **satp** register. The program should follow Sv48-paging (refer to Figure 1)
4. After proper initialization, the program should switch its execution from **Supervisor Mode** to **User Mode**. It should then start executing the user code provided.
5. The addresses during the execution of the user code should correspond to **virtual addresses** instead of the physical addresses present in the dump file.

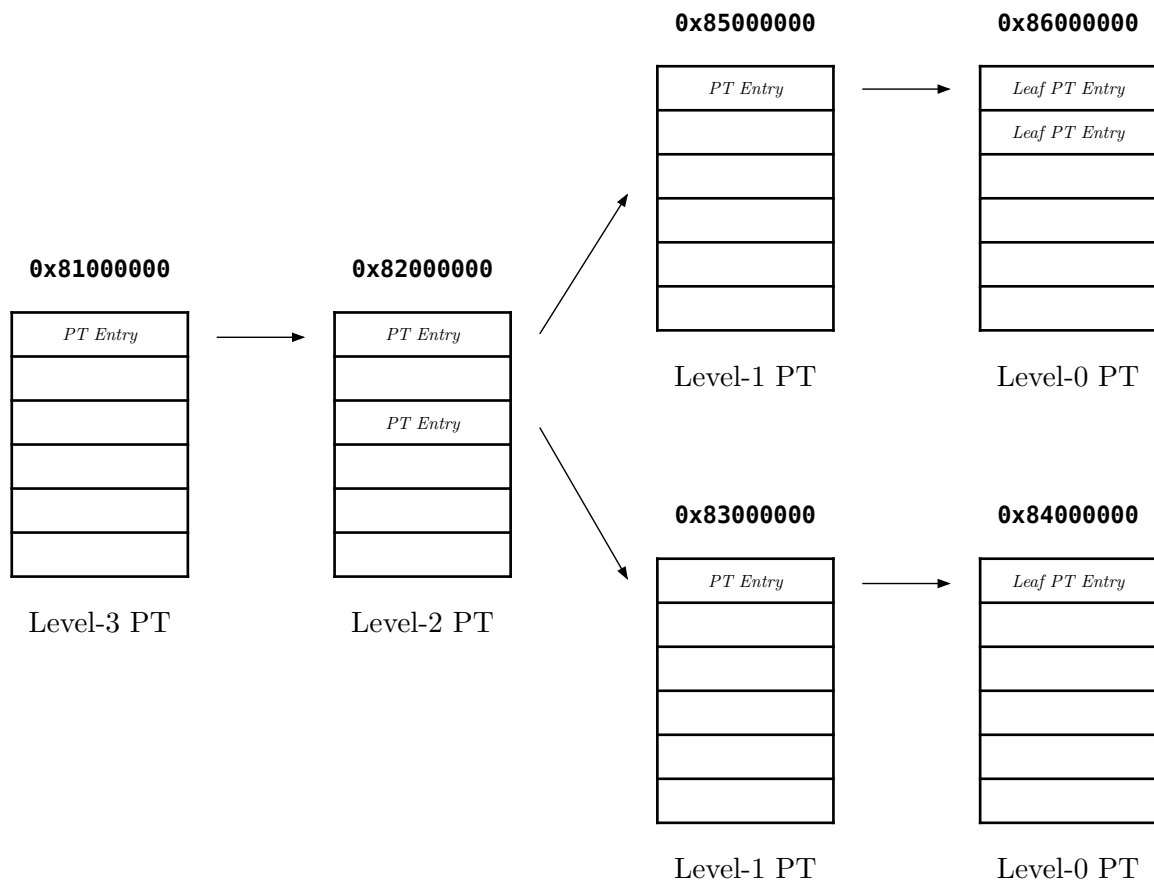


Figure 1: Page Table Structure for Sv48 (Only for Illustration purposes, actual mappings might vary depending on your roll number)

Instructions:

1. A template code has been provided, which has relevant snippets and variables with self-explanatory comments. You can use it to get started with the task.
2. The mapping done between the pages should adhere to the following format: Entry point in the linker should be `0xYZ000000` , where YZ is the hexadecimal representation of the sum of the last two digits of your roll number and 128.

For example, if your roll number is CS23B069, then $YZ = \text{hex}(69 + 128) = \text{hex}(197) = \text{C5}$. Therefore, the entry point should be `0xC5000000`. Hence, the linker.ld file will look like

```
./linker.ld
1  ENTRY(main)
2
3  SECTIONS
4  {
5      . = 0xC5000000;
6  }
```

3. The page table entries should follow appropriate conventions.
4. The user code execution should start from the virtual address `0x00000000` . The data section should be mapped immediate to the text section.

Hints:

1. The address translation starts just after the instruction where the `satp` register gets configured, but the program is still in the supervisor mode. What address should the next instruction be mapped to for its proper execution?
2. Try writing down the offsets, mapping and hand simulating the translation process for few memory addresses.
3. Be careful while setting the page table entries, especially the last few bits of it.
4. The biggest hint and the answer for the first hint is the following diagram:

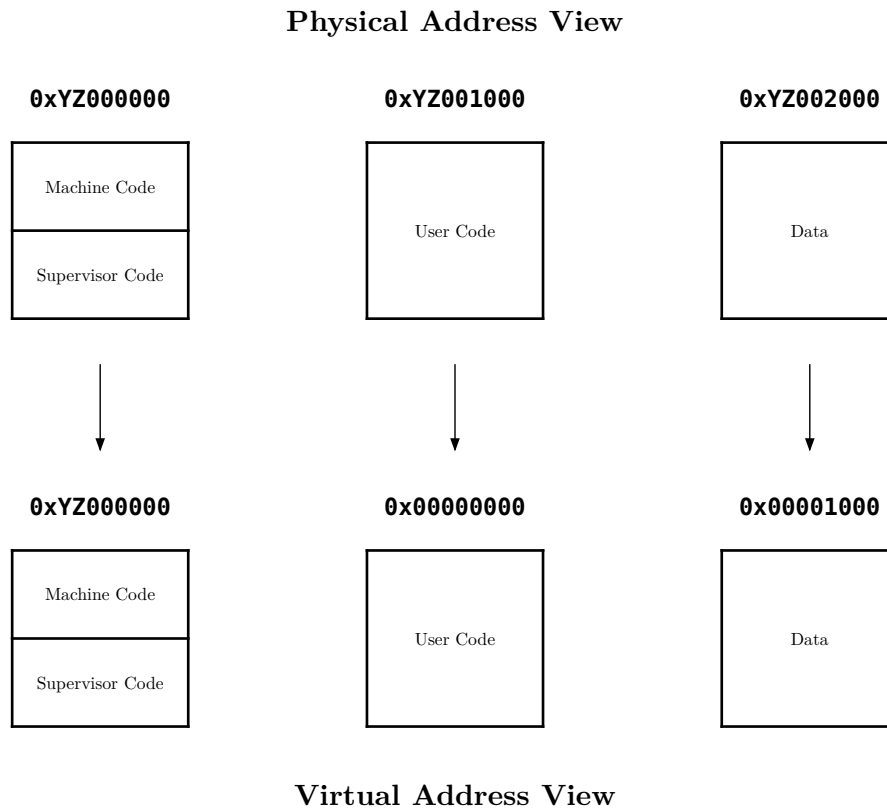


Figure 2: Pictorial view of the aim of the assignment

Bonus Task:

Implement the above task for Sv39: Page-Based 39-bit Virtual Memory System and Sv57: Page-Based 57-bit Virtual Memory System.